Project Planning and Management

Digital Egypt Pioneers Initiative (DEPI) | Data Engineering Final Project

Healthcare Data Pipeline Project

Prepared by Moataz Alaa, Youssef Mahmoud, Mohamed Mounir, Ahmed Atef, and Ahmed Yakout

Program: DEPI | Instructor: Mohamed Hamed

1. Project Overview

This project will build a small but complete healthcare data pipeline for learning and demonstration. The team will collect health data from more than one source, clean and validate it, load it into a manageable data warehouse, and present useful results in Power BI. The first version should stay simple enough for a beginner team to finish and explain clearly.

2. Recommended Technical Direction

Our recommendation is to use ETL for the first version. Python will extract and clean the CSV files, SQL Server will store the Bronze, Silver, and Gold layers, and Power BI Desktop will connect to the gold layer. This choice is easier to test locally, easier to explain in a presentation, and fits the current team level. Fabric and Databricks are good future options, but using them now could add cloud accounts, permissions, cost, and new concepts before the core pipeline is stable.

Decision	First version	Possible future change
Pipeline style	ETL: clean before loading the final tables	ELT inside Fabric or Databricks when data volume grows
Data sources	Existing health, supplements, experiment, and profile CSV files	Add Synthea CSV or FHIR data if time allows
Warehouse	SQL Server Developer or Express on a team machine	Azure SQL Database or Fabric Warehouse
Processing	Python and pandas, with SQL validation	Fabric Data Factory, notebooks, dbt, or Spark
Dashboard	Power BI Desktop report	Power BI Service or a Fabric workspace

3. Sources

The project is currently considering two data-source options.

The first option is to use the existing health and supplement dataset. This dataset contains:

- user_profiles.csv
- user_health_data.csv
- supplement_usage.csv
- experiments.csv

These files can represent two conceptual source systems:

- A health or wearable application containing user profiles and daily health measurements.
- A supplement or experiment application containing supplement usage and experiment information.

The second option is to create or use another synthetic healthcare dataset, such as Synthea. Synthea can provide synthetic patient histories, encounters, medications, allergies, and other healthcare records in formats such as CSV.

The final source option will be selected after the team compares:

- How well the data matches the project goals.
- How easy it is to clean and transform.
- Whether the data can demonstrate multiple source systems.
- How much time the team has.
- Whether the data can support useful Power BI dashboards.

At this stage, the existing CSV dataset will be used as a working option for design and testing. However, the project may switch to another synthetic dataset if the team decides that it better matches the final healthcare requirements.

The project will not use real data. All selected records will be used only for educational purposes.

4. Data Warehouse Design

Layer	Purpose	Example tables
Bronze	Store source data as received, together with source name, batch ID, and load time.	Raw source tables based on the selected dataset
Silver	Clean and standardize dates, data types, missing values, duplicates, units, and invalid records.	Cleaned patient, health, supplement, encounter, or other selected source tables
Gold	Store business-ready fact and dimension tables for Power BI reporting.	Patient, date, experiment, supplement, encounter, health, or other dimensions and facts

5. Data Processing Tools

Python and pandas will handle extraction, cleaning, standardization, and repeatable validation. SQL Server will handle table creation, keys, joins, and final analytical views. Power Query can be used for small presentation-side shaping, but the main cleaning rules should stay in Python or SQL so they can be repeated and tested.

- Python: read CSV files, standardize column names, parse dates, normalize units such as mg and milligrams, detect duplicates, validate ranges, and write clean/rejected outputs.
- SQL Server: load bronze, silver, and gold tables; add surrogate keys; enforce relationships; store load timestamps and batch IDs; run quality checks.
- Power BI: connect to gold tables, create measures, define relationships, and build the final report.
- Git and a simple README: keep the ETL code, SQL scripts, data dictionary, screenshots, and run instructions together.

6. Project Plan

Duration	Main work	Deliverable
1	Confirm scope, sources, team roles, and success measures.	Approved plan and requirements
2	Profile the CSV files and document columns, keys, and data problems.	Source data dictionary and profiling notes
3	Build Bronze tables and a repeatable ingestion script.	Raw load working with batch metadata
4	Build Silver cleaning and rejection rules.	Clean tables, rejects, and quality report
5	Build Gold star schema and SQL views.	Dimensions, facts, keys, and test queries
6	Connect Power BI, create measures, and report pages.	First dashboard version
7	Evaluate totals, relationships, refresh, and edge cases.	Evaluate evidence and corrected dashboard
8	Prepare documentation, presentation, and final demonstration.	Final project package

7. Task Assignment and Roles

Team member	Main responsibility	Backup responsibility
Ahmed Atef (Team Leader)	Project leadership and Power BI dashboard	Manage the team, follow up on progress, and dashboards and collect meeting notes
Moataz Alaa	SQL Server data warehouse and data model	Create warehouse layers, tables, relationships, and SQL validation queries
Youssef Mahmoud	Meeting notes, documentation, and data engineering support	Update documents, support Power BI, data cleaning, SQL, research, and other tasks
Mohamed Mounir	Project planning and data engineering guidance	Support planning, requirements, tool selection, and technical decisions
Ahmed Yakout	Testing, risk tracking, and data dictionary support	Support testing, data-quality records, risks, and documentation

Flexibility of Roles:

These roles are not permanent restrictions. The team may change, exchange, or add responsibilities at any time according to project requirements, member skills, availability, and work progress. All members are expected to cooperate and support other tasks when needed.

8. Risks and Mitigation

Risk	Effect	Mitigation
The team chooses too many technologies.	The project becomes difficult to finish.	Use Python, SQL Server, and Power BI first. Record Fabric and Databricks as future options.
CSV files contain missing or inconsistent values.	Wrong totals or failed loads.	Keep raw rows, send invalid rows to etl_rejects, and publish quality counts.
IDs do not match between source systems.	Rows disappear during joins.	Use a default “Unknown” member to handle orphaned records, and report orphan counts.
Power BI relationships duplicate measures.	Incorrect dashboard totals.	Use a star schema and test each measure against SQL totals.
One team member becomes a single point of failure.	Work stops when that person is unavailable.	Use shared scripts, clear naming, and pair review for important work.
Cloud setup or account access is delayed.	The team loses time on infrastructure.	Finish a local SQL Server version first and move it to cloud only if it helps.

9. Project KPIs

KPI	Target for the project	How it will be checked
Source coverage	All source files load successfully.	Ingestion log and row counts
Data quality visibility	Every rejected row has a reason.	Reject table and quality report
Reconciliation accuracy	Gold totals match approved silver totals.	SQL reconciliation queries
Pipeline repeatability	The same run gives the same result.	Run the pipeline twice with the same batch
Dashboard usability	A user can answer the agreed questions without help.	Team walkthrough and feedback
Documentation completeness	Another student can run and explain the project.	README and presentation checklist
Pipeline execution time	The full ETL pipeline finishes in a reasonable time.	Log the start and end times of the Python and SQL scripts to measure total execution duration
Dashboard refresh time	The Power BI dashboard refreshes and renders visuals in a reasonable time.	Use the Power BI Performance Analyzer or a manual timer

10. Change and Improvement Plan

The project is allowed to change after the first working version. A change should be accepted when it improves the result without putting the deadline at risk. The team should record the reason, expected benefit, extra work, and effect on the current design. If the local SQL Server solution is stable, moving to Azure SQL or Fabric can be presented as a future deployment path instead of replacing the working version at the last minute.

References Used

- MITRE Synthea official site: https://synthetichealth.github.io/synthea/
- MITRE Synthea downloads and formats: https://synthetichealth.github.io/downloads.html
- Microsoft Fabric overview: https://learn.microsoft.com/en-us/fabric/fundamentals/microsoft-fabric-overview
- Fabric Data Warehouse: https://learn.microsoft.com/en-us/fabric/data-warehouse/data-warehousing

- Azure SQL Database documentation: https://learn.microsoft.com/en-us/azure/azure-sql/database
- Power BI star schema guidance: https://learn.microsoft.com/en-us/power-bi/guidance/star-schema
- Power BI overview: https://learn.microsoft.com/en-us/power-bi/fundamentals/power-bi-overview

DEPI